UNIVERSIDADE FEDERAL DO CEARÁ

Test Smells em projetos OpenSource

Disciplina: Qualidade de Software
Equipe: Antonio Felipe Lima Rodrigues (565195) e Lizandra Mara de Souza Pinheiro (566175)
Projeto: PR-Agent, https://github.com/The-PR-Agent/pr-agent
Fork: https://github.com/lizandrapinheiro/pr-agent
Pull Request: https://github.com/The-PR-Agent/pr-agent/pull/4440 (mesclado em 28/10/2026)
Métricas: https://github.com/lizandrapinheiro/pr-agent/tree/metricas

Visão geral

Smell	Responsável	Ocorrências	Commit	TEMPY no projeto (antes e depois)
Exception Handling	Lizandra Pinheiro	10 (EH-01 a EH-10)	16aa165c	269 e 259
Verbose Test	Lizandra Pinheiro	10 (VT-01 a VT-10)	6d58b122	77 e 65
Conditional Test Logic	Antonio Felipe	10 (CTL-01 a CTL-10)	368f4672 e 80277fa	508 e 496
Unknown Test	Antonio Felipe	10 (UT-01 a UT-10)	28ceba43	142 e 134

UNIVERSIDADE FEDERAL DO CEARÁ

1. Exception Handling

Resumo

Responsável	Lizandra Mara de Souza Pinheiro
Branch	refactor/remove-test-smell-exception-handling
Commit	https://github.com/lizandrapinheiro/pr-agent/commit/16aa165c
Eu estou atualmente trabalhando na refatoração do seguinte code smell:	Exception Handling. Segundo o CATSPY, ele ocorre quando o teste usa blocos try/except (ou try/finally) para controlar exceções na mão, em vez de usar os recursos do framework de testes. O TEMPY detecta qualquer método de teste que tenha um bloco try.

Ocorrências refatoradas

ID	Arquivo (tests/unittest/)	Teste	Linha(s)	Refatoração aplicada
EH-01	test_inline_comment_dedup.py	test_iter_unsupported_provider_raises	166	try/except trocado por pytest.raises
EH-02	test_code_suggestions_comment_identity.py	test_custom_suggestions_heading_changes_presentation_only	20	try/finally trocado pela fixture restore_suggestions_heading
EH-03	test_code_suggestions_comment_identity.py	test_suggestions_heading_is_trimmed	36	try/finally trocado pela fixture restore_suggestions_heading
EH-04	test_code_suggestions_comment_identity.py	test_invalid_suggestions_heading_falls_back_to_default	56	try/finally trocado pela fixture restore_suggestions_heading
EH-05	test_code_suggestions_comment_identity.py	test_summarized_artifact_uses_custom_heading_without_identity	92	try/finally trocado pela fixture restore_suggestions_heading
EH-06	test_code_suggestions_comment_identity.py	test_github_check_run_receives_configured_presentation_without_identity	139	try/finally trocado pela fixture restore_check_run_setting
EH-07	test_webhook_logic_core.py	test_shared_should_process_pr_logic_rules	1312	try/finally trocado pela fixture restore_filter_settings

ID	Arquivo (tests/unittest/)	Teste	Linha(s)	Refatoração aplicada
EH-08	test_webhook_logic_core.py	test_shared_should_process_pr_logic_payload_parsing	1375	try/finally trocado pela fixture restore_filter_settings
EH-09	test_webhook_logic_core.py	test_shared_should_process_pr_logic_explicit_empty_overrides_payload	1438	try/finally trocado pela fixture restore_filter_settings
EH-10	test_webhook_logic_core.py	test_bitbucket_server_should_process_pr_logic_fails_open_on_filter_exception	1490	try/finally trocado pela fixture restore_filter_settings

EH-01: try/except com raise AssertionError manual para verificar se uma exceção é lançada, exatamente o caso descrito no CATSPY, esconde a intenção do teste. EH-02 a EH-10: try/finally usado apenas para restaurar configurações globais (Dynaconf) depois do teste. O TEMPY aponta qualquer bloco try, o bloco mistura preparação/limpeza com a lógica do teste e prejudica a legibilidade.

Registro da refatoração

Minhas principais dificuldades na remoção do code smell são:	- O exemplo do CATSPY só cobre try/except verificando exceção (assertRaises). Só a EH-01 seguia esse padrão; nas outras 9 o bloco era try/finally de limpeza, sem except. Consultei o monitor, que autorizou adaptar a refatoração desde que registrada no relatório. - Escolher ocorrências sem mudar o comportamento: muitos try/except do projeto ficam dentro de funções assíncronas simuladas (ex.: test_litellm_callback_drain.py), onde o tratamento de exceção faz parte do que é testado. Esses ficaram de fora. - Efeitos colaterais: na EH-01 o pass do except (Non-Functional Statement) saiu junto; na EH-07 e EH-09 o for de restauração no finally (Conditional Test Logic) também foi removido; EH-08 e EH-09 ficaram com menos de 45 linhas e deixaram de ser Verbose Test. - Estilo: na primeira versão as fixtures ficaram no meio dos imports e o ruff apontou erro; foi preciso movê-las para depois dos imports. - Suíte completa com 1 falha a mais que no antes; ao reexecutar só as falhas, 2 passaram, instabilidade por tempo, sem relação com a refatoração.

Usei o CATSPY para:	- Definição do smell, critério de detecção e exemplo de refatoração (assertRaises substituído por pytest.raises). Página consultada: https://cat-spy.vercel.app/smells/Exception%20Handling
Eu estou usando os seguintes métodos de refatoração para remover o code smell:	1. EH-01 (conforme o CATSPY): troca do try/except com raise AssertionError manual por with pytest.raises(NotImplementedError), equivalente no pytest ao assertRaises. 2. EH-02 a EH-10 (adaptação): a preparação e a restauração das configurações foram movidas para fixtures do pytest com yield (restore_suggestions_heading, restore_check_run_setting e restore_filter_settings). O código antes do yield guarda os valores e o código depois os restaura, mesmo se o teste falhar, a mesma garantia do finally, sem o bloco try no teste. Asserções e lógica dos testes inalteradas.

Verificação

Testes e TEMPY nos arquivos alterados	Testes dos arquivos: test_inline_comment_dedup.py 62 passed; test_code_suggestions_comment_identity.py 19 passed; test_webhook_logic_core.py 73 passed (sem falhas). TEMPY (--arquivo): Exception Handling de 27 para 26, de 5 para 0 e de 10 para 6 nos três arquivos; ocorrências removidas: sim; nenhum smell novo. ruff check: sem erros.
Suíte completa no commit	- Antes: 12.326 passed, 43 failed, 8 errors - Depois: 12.325 passed, 44 failed, 8 errors A diferença de 1 teste é instabilidade por tempo em testes de cancelamento: ao reexecutar só as falhas (--lf), 2 passaram. As falhas restantes são as mesmas do antes, específicas do Windows.
TEMPY: total deste tipo no projeto	Passou de 269 para 259, redução de 10

UNIVERSIDADE FEDERAL DO CEARÁ

2. Verbose Test

Resumo

Responsável	Lizandra Mara de Souza Pinheiro
Branch	refactor/remove-test-smell-verbose-test
Commit	https://github.com/lizandrapinheiro/pr-agent/commit/6d58b1229c5adbe530de86192ed703f46c95f4ae
Eu estou atualmente trabalhando na refatoração do seguinte code smell:	Verbose Test. Segundo o CATSPY, ele ocorre quando um método de teste é excessivamente longo, o que indica múltiplas responsabilidades e dificulta entender e manter o teste. O TEMPY aponta testes com 45 linhas ou mais.

Ocorrências refatoradas

ID	Arquivo (tests/unittest/)	Teste	Linha(s)	Refatoração aplicada
VT-01	test_per_directory_settings.py	test_description_questions_and_similar_issue_host_only_keys_are_dropped	637 a 747 (111 linhas)	Dividido em 6 testes, preparação na fixture host_only_keys_applied
VT-02	test_codecommit_client.py	test_get_differences	21 a 94 (74 linhas)	Dividido em 5 testes, chamada na fixture differences
VT-03	test_webhook_logic_core.py	test_shared_should_process_pr_logic_rules	1300 a 1366	Dividido em 6 testes, um por regra, com a fixture empty_filter_settings
VT-04	test_get_max_tokens.py	test_claude_model_family_metadata_key_validation	1155 a 1212 (58 linhas)	Dividido em 5 testes, dados inválidos em fixtures
VT-05	test_output_models.py	test_required_label_and_list_constraints_are_enforced	195 a 248 (54 linhas)	16 blocos repetidos trocados por parametrize
VT-06	test_codecommit_provider.py	test_add_additional_newlines	1461 a 1509 (49 linhas)	Dividido em 4 testes, um por cenário

ID	Arquivo (tests/unittest/)	Teste	Linha(s)	Refatoração aplicada
VT-07	test_get_max_tokens.py	test_claude_opus_4_6_family_expansion_and_capabilities	1106 a 1153 (48 linhas)	Dividido em 4 testes, dados em atributos da classe
VT-08	test_load_yaml.py	test_load_invalid_yaml1	290 a 336 (47 linhas)	Dividido em 2 testes, YAML em atributo da classe
VT-09	test_convert_to_markdown.py	test_can_be_split	239 a 296 (58 linhas)	Entrada e saída esperada extraídas para atributos da classe
VT-10	test_bitbucket_provider.py	test_get_diff_files_splits_crlf_headers_after_preamble_in_diffstat_order	561 a 607 (47 linhas)	Texto do diff extraído para atributo da classe

Testes com 45 linhas ou mais. Em VT-01 a VT-07 o teste verificava várias responsabilidades (grupos de asserts separados por comentários, vários cenários ou vários casos inválidos); em VT-08 a VT-10 o tamanho vinha principalmente de dados de entrada e saídas esperadas escritos dentro do teste.

Registro da refatoração

Minhas principais dificuldades na remoção do code smell são:	- Priorizamos os testes mais longos (mais severos), mas só aqueles em que a divisão fosse completa e não mudasse o que o teste verifica. - A refatoração do CATSPY (dividir o teste) nem sempre se aplica: em VT-09 e VT-10 o teste tinha uma única responsabilidade e era longo só pelos dados; nesses casos extraí os dados para atributos de classe. - O TEMPY também conta funções auxiliares longas: os dados grandes não podiam ficar dentro de fixtures (virariam um smell novo), então foram para atributos de classe ou constantes de módulo. - Dependência de ordem (VT-03): cada bloco zerava a configuração para o próximo; criei a fixture empty_filter_settings para garantir o mesmo estado inicial em cada teste. - Smells novos expostos pela divisão: em VT-04 o trecho que valida as famílias atuais já não tinha asserção e virou Unknown Test; em VT-08 o teste do safe_load usa só

	pytest.raises, que o TEMPY não reconhece (falso positivo). Não refatorados, conforme o enunciado. - Sobreposição com o colega: VT-04 e VT-07 têm um for (Conditional Test Logic); mantive o for intacto num teste próprio. - Erros de cópia: num caso sobrou o cabeçalho da função antiga e o pytest não carregou o arquivo. - As divisões do VT-04 e do VT-08 fizeram o TEMPY apontar dois Unknown Test em testes novos (test_shipped_claude_model_families_pass_validation e test_invalid_yaml1_is_rejected_by_safe_load). Analisamos e decidimos não refatorar: o enunciado permite, o primeiro trecho já não tinha asserção no teste original e o segundo verifica a exceção com pytest.raises, que o TEMPY não reconhece (falso positivo). Uma correção exigiria uma asserção artificial. A justificativa completa está no relatório final, seção 5.
Usei o CATSPY para:	- Definição, efeitos negativos e exemplo de refatoração (dividir o teste em testes menores, cada um com uma responsabilidade). Página consultada: https://cat-spy.vercel.app/smells/Verbose%20Test
Eu estou usando os seguintes métodos de refatoração para remover o code smell:	1. Dividir o teste em testes menores por responsabilidade, como no CATSPY (VT-01 a VT-04, VT-06, VT-07), com a preparação comum em fixtures (host_only_keys_applied, differences, empty_filter_settings, family_with_unknown_top_level_key, family_with_unknown_nested_alias_key). 2. Parametrização (VT-05): 16 blocos with pytest.raises(ValueError) repetidos viraram um @pytest.mark.parametrize com a lista _INVALID_MODEL_PAYLOADS. 3. Extrair dados para fora do teste (VT-08 a VT-10, e os dados de VT-01 e VT-02): entradas e saídas esperadas movidas para atributos de classe. Em todos os casos os asserts e os valores comparados não mudaram.

Verificação

Testes e TEMPY nos arquivos alterados	Testes dos arquivos, antes e depois: per_directory_settings de 69 para 74; codecommit_client de 9 para 13; webhook_logic_core de 73 para 78; get_max_tokens de 837 para 844; output_models de 41 para 56; codecommit_provider

	de 57 para 60; load_yaml de 63 para 64; convert_to_markdown de 31 para 31; bitbucket_provider de 152 para 152 (sem falhas). TEMPY (--arquivo): Verbose Test removido em todas as 10 ocorrências: sim. ruff check: sem erros.
Suíte completa no commit	- Antes: 12.326 passed, 43 failed, 8 errors - Depois: 12.362 passed, 47 failed, 8 errors Os +40 testes vêm das divisões. As 4 falhas a mais estão todas em test_litellm_callback_drain.py (arquivo não alterado), em testes de cancelamento sensíveis a tempo; com git stash confirmei que o arquivo falha da mesma forma sem as refatorações. Nenhuma falha nos 9 arquivos refatorados.
TEMPY: total deste tipo no projeto	Passou de 77 para 65 (10 a menos por esta refatoração e 2 a menos como efeito colateral do Exception Handling)

UNIVERSIDADE FEDERAL DO CEARÁ

3. Conditional Test Logic

Resumo

Responsável	Antonio Felipe Lima Rodrigues
Branch	refactor/remove-test-smell-conditional-test-logic
Commit	https://github.com/lizandrapinheiro/pr-agent/commit/368f4672 (commit das 10 ocorrências) https://github.com/lizandrapinheiro/pr-agent/commit/80277fa8 (correção após a revisão do PR)
Eu estou atualmente trabalhando na refatoração do seguinte code smell:	Conditional Test Logic. Segundo o CATSPY, este problema ocorre devido à presença de métodos de teste que contém instruções condicionais, por exemplo, a utilização de if-else ou loops como for ou while. Métodos de teste que incluem instruções condicionais podem comprometer a eficácia dos testes. O TEMPY detecta métodos de teste que apresentam essas estruturas em seu código.

Ocorrências refatoradas

ID	Arquivo (tests/unittest/)	Teste	Linha(s)	Refatoração aplicada
CTL-01	test_effort_bar_clamp.py	test_the_bar_is_always_five_segments	37	for trocado por @pytest.mark.parametrize
CTL-02	test_docker_hardening.py	test_dockerignore_lists_both_supported_secret_files	52	for trocado por @pytest.mark.parametrize
CTL-03	test_diff_pipeline_core.py	test_deletion_only_hunk_is_rendered_when_it_is_the_last_hunk	704	for trocado por @pytest.mark.parametrize
CTL-04	test_diff_pipeline_core.py	test_deletion_only_hunk_is_rendered_when_another_hunk_follows	711	for trocado por @pytest.mark.parametrize (valor +added restaurado em 80277fa)
CTL-05	test_extend_patch.py	test_single_hunk	60	for trocado por @pytest.mark.parametrize
CTL-06	test_extend_patch.py	test_single_line_hunk_does_not_resurrect_the_deleted_line	280	for trocado por @pytest.mark.parametrize (espaços

ID	Arquivo (tests/unittest/)	Teste	Linha(s)	Refatoração aplicada
				iniciais restaurados em 80277fa)
CTL-07	test_extra_config_url.py	test_resolve_rejects_non_string_source	795	for trocado por @pytest.mark.parametrize
CTL-08	test_command_descriptions.py	test_cli_and_bot_help_use_canonical_command_descriptions	20	for trocado por @pytest.mark.parametrize
CTL-09	test_file_filter.py	test_every_supported_platform_actually_drops_a_matching_file	716	for trocado por @pytest.mark.parametrize
CTL-10	test_file_filter.py	test_supported_platforms_do_not_warn	727	for trocado por @pytest.mark.parametrize

As 10 ocorrências apresentavam laços for responsáveis por executar asserções repetidas sobre diferentes valores. Nas CTL-01 a CTL-07, os laços percorriam listas ou tuplas de valores fixos, enquanto nas CTL-08 a CTL-10 percorriam coleções definidas no módulo. Em ambos os casos, múltiplas verificações eram concentradas em uma única execução do teste.

Registro da refatoração

Minhas principais dificuldades na remoção do code smell são:	Não foram identificadas dificuldades significativas durante a execução inicial das refatorações. Entretanto, a revisão do Pull Request apontou dois problemas relacionados à preservação das verificações originais. Durante a parametrização do CTL-06, os valores “ l2”, “ l3” e “ l4” perderam seus espaços iniciais. Como esses espaços identificam linhas de contexto em um diff, sua ausência enfraquecia as verificações, permitindo que o teste fosse aprovado mesmo diante de linhas adicionadas ou removidas incorretamente. Durante a substituição do laço no CTL-04, o valor “+added” não foi incluído na lista de parâmetros, deixando uma das verificações originais sem execução.
Usei o CATSPY para:	Consultar a definição do Conditional Test Logic e a recomendação de substituir iterações internas por testes parametrizados. Página consultada: https://cat-spy.vercel.app/smells/Conditional%20Test%20Logic

Eu estou usando os seguintes métodos de refatoração para remover o code smell:	1. Parametrização de testes (CTL-01 a CTL-10): substituição dos laços for por @pytest.mark.parametrize, utilizando listas de valores fixos (CTL-01 a CTL-07) e coleções definidas no módulo (CTL-08 a CTL-10). Cada valor passou a ser verificado em uma execução independente, preservando as asserções e os dados originais dos testes. O TEMPY sinaliza métodos nos quais não reconhece asserções explícitas.

Verificação

Testes e TEMPY nos arquivos alterados	Testes dos arquivos, antes e depois: effort_bar_clamp de 9 para 14; docker_hardening de 14 para 15; diff_pipeline_core de 44 para 49; extend_patch de 41 para 45; extra_config_url de 42 para 45; command_descriptions de 10 para 12; file_filter de 109 para 123 (sem falhas). TEMPY (--arquivo), antes e depois dos CTL: effort_bar_clamp de 1 para 0; docker_hardening de 1 para 0; diff_pipeline_core de 8 para 6; extend_patch de 4 para 2; extra_config_url de 8 para 7; command_descriptions de 1 para 0; file_filter de 6 para 4. Conditional Test Logic removido em todas as 10 ocorrências. ruff check: sem erros.
Suíte completa no commit	- Antes: 12.362 passed, 47 failed, 8 errors - Depois: 12.392 passed, 50 failed, 8 errors Nenhum dos 7 arquivos alterados consta nas falhas após a refatoração. O aumento de execuções é compatível com a parametrização e a variação de falhas não foi atribuída às alterações.
TEMPY: total deste tipo no projeto	Passou de 508 para 496 (10 a menos por esta refatoração e 2 a menos como efeito colateral do Exception Handling). Contagem global dos demais tipos inalterada.

UNIVERSIDADE FEDERAL DO CEARÁ

4. Unknown Test

Resumo

Responsável	Antonio Felipe Lima Rodrigues
Branch	refactor/remove-test-smell-unknown-test
Commit	https://github.com/lizandrapinheiro/pr-agent/commit/28ceba43
Eu estou atualmente trabalhando na refatoração do seguinte code smell:	Unknown Test. Segundo o CATSPY, ocorre quando um método de teste é executado sem declarações de asserção claras. Nesse caso, o teste será aprovado se nenhuma exceção for lançada durante a execução. Essa abordagem pode dificultar a compreensão dos objetivos do teste, já que não há verificações explícitas para validar as condições esperadas do código sendo testado.

Ocorrências refatoradas

ID	Arquivo (tests/unittest/)	Teste	Linha(s)	Refatoração aplicada
UT-01	test_secret_provider_factory.py	test_accepts_supported_providers	17	Asserção explícita adicionada sobre o resultado
UT-02	test_secret_provider_factory.py	test_accepts_unset_provider	21	Asserção explícita adicionada sobre o resultado
UT-03	test_mosaico_metadata.py	test_never_raises	40	Asserção explícita adicionada sobre o resultado
UT-04	test_mosaico_metadata.py	test_none_meta_is_noop	53	Indicador de execução do bloco e asserção explícita
UT-05	test_mosaico_metadata.py	test_log_context_partial_and_empty	73	Indicador de execução do bloco e asserção explícita
UT-06	test_mosaico_env_bridge.py	test_empty_meta_is_clean_passthrough	218	Indicador de execução do bloco e asserção explícita
UT-07	test_custom_merge_loader_security.py	test_safe_pr_agent_config_does_not_raise	127	Asserção explícita adicionada sobre o resultado
UT-08	test_config_loader_secrets.py	test_apply_secrets_manager_config_no_provider	27	Asserção explícita adicionada sobre o resultado

ID	Arquivo (tests/unittest/)	Teste	Linha(s)	Refatoração aplicada
UT-09	test_config_loader_secrets.py	test_apply_secrets_manager_config_exception_handling	114	Asserção explícita adicionada sobre o resultado
UT-10	test_push_outputs.py	test_errors_are_non_fatal	278	Asserção explícita adicionada sobre o resultado

As 10 ocorrências apresentavam testes que executavam operações sem realizar verificações explícitas sobre seus resultados. Nas UT-01, UT-02 e UT-07 a UT-10, os testes apenas executavam funções para confirmar que determinadas operações não lançavam exceções. Na UT-03, diferentes entradas eram processadas sem verificar os valores retornados. Já nas UT-04 a UT-06, os testes utilizavam blocos de contexto sem asserções, verificando apenas implicitamente se sua execução ocorria normalmente.

Registro da refatoração

Minhas principais dificuldades na remoção do code smell são:	A principal preocupação foi introduzir asserções que preservassem o comportamento originalmente verificado pelos testes, sem acrescentar novos cenários ou ampliar o escopo das verificações. Nas UT-04 a UT-06, os testes verificavam a execução de blocos de contexto sem exceções. Foram introduzidos indicadores de execução para tornar essa verificação explícita, mantendo a finalidade original. Na UT-05, a substituição de duas instruções pass também eliminou duas ocorrências de Non-Functional Statement. Essa remoção foi considerada um efeito incidental da refatoração, e não um segundo objetivo. Dificuldade na identificação de ocorrências válidas de Unknown Test, pois o TEMPY também classificava testes que já possuíam asserções explícitas ou verificações indiretas de comportamento. Foi necessário analisar essas ocorrências para distinguir falsos positivos de testes que realmente não apresentavam verificações.
Usei o CATSPY para:	Consultar a definição do Unknown Test, compreender seus efeitos negativos e orientar a introdução de verificações explícitas em testes que não apresentavam asserções, respeitando o comportamento originalmente testado. Página consultada: https://cat-spy.vercel.app/smells/Unknown%20Test
Eu estou usando os seguintes métodos de	1. Introdução de asserções explícitas (UT-01 a UT-10): inclusão de verificações sobre os resultados das operações já

refatoração para remover o code smell:	executadas. Nos testes que verificavam a ausência de exceções, foram utilizadas asserções sobre o retorno das funções ou indicadores de execução dos blocos de contexto. As alterações foram restritas à remoção do Unknown Test, sem modificar os cenários originalmente testados.

Verificação

Testes e TEMPY nos arquivos alterados	Testes dos arquivos: test_secret_provider_factory.py 10 passed; test_mosaico_metadata.py 18 passed; test_mosaico_env_bridge.py 17 passed; test_custom_merge_loader_security.py 83 passed; test_config_loader_secrets.py 8 passed; test_push_outputs.py 53 passed (sem falhas). TEMPY (--arquivo), antes e depois dos UT: secret_provider_factory de 5 para 3; mosaico_metadata de 3 para 0; mosaico_env_bridge de 1 para 0; custom_merge_loader_security de 9 para 8; config_loader_secrets de 2 para 0; push_outputs de 2 para 1. Unknown Test removido em todas as 10 ocorrências. Remoções incidentais: 3 Non-Functional Statement removidos, sendo 2 em test_mosaico_metadata.py e 1 em test_mosaico_env_bridge.py, como consequência das refatorações de Unknown Test. Nenhum smell novo. ruff check: sem erros.
Suíte completa no commit	- Antes: 12.392 passed, 50 failed, 8 errors - Depois: 12.393 passed, 49 failed, 8 errors Foi observada uma redução de uma falha na suíte completa. Nenhum dos 6 arquivos refatorados apresentou falhas antes ou depois das alterações. A diferença pode estar relacionada à instabilidade de testes preexistentes, não sendo possível atribuí-la diretamente às refatorações.
TEMPY: total deste tipo no projeto	Passou de 142 para 134 (10 a menos por esta refatoração e 2 adicionadas incidentalmente por refatorações anteriores, fora do nosso escopo de correção nesta refatoração). Foram observadas remoções incidentais de Non-Functional Statement, decorrentes da substituição de instruções pass por verificações explícitas.